IGA infrastructure and connector setup

JBC Cybersecurity Enterprise IGA Lab

I built the infrastructure and HR connector foundation for my Identity Governance and Administration (IGA) lab. This walkthrough follows the work from separating the engine and database hosts to validating a synthetic HR feed, storing identity records and testing reconciliation. Reconciliation compares incoming HR data with saved identity state to identify new or changed records.

This project is an independent IGA simulation modeled on common enterprise identity-governance workflows. It was not built in a Saviynt tenant.

Choose a website subsection

Website subsection	Coverage	PDF pages
VM and engine setup	Host roles, Python environment and repository baseline	2
PostgreSQL setup	Connectivity, database role, identity tables and audit structure	3–4
HR connector setup	Validation, reconciliation, persistence and captured tests	5–6

The captured results show six identities stored in PostgreSQL, invalid employee identifiers rejected and a single job-title change detected. This milestone established the identity-data foundation for later provisioning workflows.

The screenshots record prior lab execution. I did not rerun the tests while preparing this walkthrough. Missing or unsafe captures are identified where they would support a step.

VM and engine setup

Step 1 Separate the host responsibilities

I used iga-engine-01 for the Python processing code and iga-db-01 for PostgreSQL. The engine reads and evaluates the HR feed; the database retains identity state and audit events. This separation created a clear connection boundary to configure and test.

I also checked connectivity to the application host. The successful ping established network reachability only. Application service readiness was a separate milestone.

Step 2 Prepare the engine workspace

I organized the repository into source, configuration, data, logs, tests and evidence folders. I initialized Git, created the initial commit and checked the working tree. The captures show a clean working tree at that checkpoint.

Step 3 Prepare the Python environment

I used a Python virtual environment and installed the connector dependencies. Figure 1 shows psycopg for PostgreSQL access, pydantic for data modeling and validation, and pytest for testing. The package listing verified that those dependencies were installed.

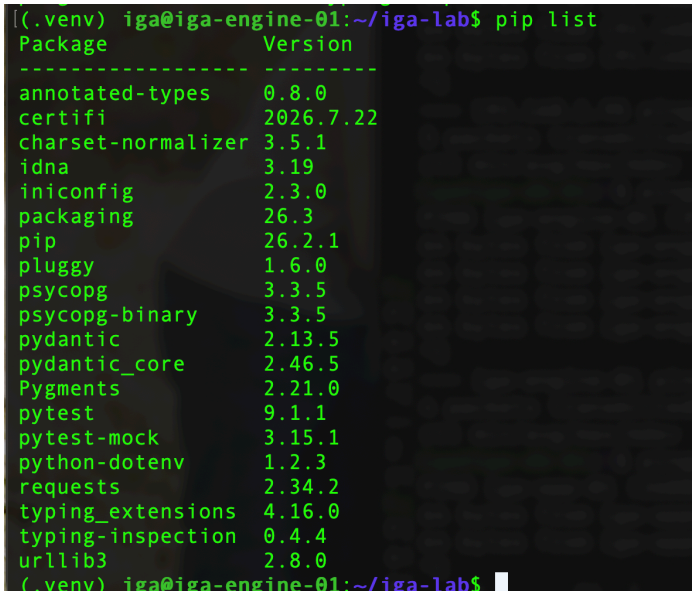


Figure 1. The engine virtual environment contains the database, validation and testing dependencies.

VM resource allocations and installation screens were not captured. The package inventory does not establish a locked dependency build. The repository captures also do not constitute a complete secret scan.

PostgreSQL connectivity and access

Step 4 Configure and check database connectivity

I configured PostgreSQL to listen on loopback and a selected host address. I allowed TCP port 5432 from the engine address in the database firewall, then checked the service and opened a remote psql session from the engine.

In that session, I queried the current database, current user, and client and server addresses. The connection reported TLS 1.3 and returned SQL results. This verified working database access more directly than the service wrapper status, which showed active (exited).

[SCREENSHOT NEEDED: Sanitized engine-to-database psql session showing TLS 1.3 and successful SQL results. Hide client and server addresses and private connection details.]

The firewall capture also showed broader SSH access. Restricting database traffic did not establish that every administrative connection was restricted.

[SCREENSHOT NEEDED: Sanitized database firewall rules. Hide the engine address consistently while retaining port 5432 and the broader SSH rules.]

Step 5 Create the lab database and engine role

I created the iga_lab database and a dedicated iga_engine role. Figure 2 shows no elevated role attributes listed for iga_engine, while postgres appears separately with administrative attributes.

```
iga_lab=# \du
                             List of roles
 Role name  |                         Attributes
------------+------------------------------------------------------------
 iga_engine |
 postgres   | Superuser, Create role, Create DB, Replication, Bypass RLS

iga_lab=#
```

Figure 2. The engine role has no listed superuser attributes; the administrative postgres role is shown separately.

The engine role owned the application tables. Separating schema ownership from runtime access and reviewing effective grants remain hardening tasks. The role listing alone does not verify least privilege.

PostgreSQL identity and audit model

Step 6 Create the identity and audit tables

I created the five PostgreSQL tables shown in Figure 3. They separated people, platform accounts, entitlements and account-to-access relationships. An entitlement is an assignable group, role or permission.

```
iga_lab=> \dt
                List of relations
 Schema |         Name          | Type  |   Owner
--------+-----------------------+-------+------------
 public | account_entitlements | table | iga_engine
 public | accounts             | table | iga_engine
 public | audit_events         | table | iga_engine
 public | entitlements         | table | iga_engine
 public | identities           | table | iga_engine
(5 rows)

iga_lab=>
```

Figure 3. PostgreSQL contains the five tables used for identity state, access relationships and audit events.

Table	Purpose in the model
identities	Stores the person and authoritative HR attributes.
accounts	Represents a person’s accounts on connected systems.
entitlements	Catalogs assignable groups, roles or permissions.
account_entitlements	Represents the association between accounts and access.
audit_events	Records processing actions and outcomes.

Step 7 Add fields for traceable processing

I added audit fields for timestamps, correlation IDs, actors, employee IDs, event types, requested actions, outcomes and JSON details. The schema captures also show indexes for correlation, employee and event-type lookup.

A correlation ID links records from the same processing run. I later checked a RECONCILIATION_COMPLETE event with SUCCESS and matched its correlation ID to the run output. Additional captured output linked the run ID and source hash to the audit details, allowing the processed input and recorded outcome to be traced together.

Creating these tables and audit fields did not prove every relationship constraint, transaction boundary or backup policy. Audit-write protection also needs a separate implementation and verification step.

HR connector setup

Step 8 Build and validate the HR input

I used employees.csv as the simulated authoritative HR source, the input the engine trusts for identity attributes. I added HR modeling and validation modules and used the database connector to access saved identity state.

I used employeeID as the stable person key. Email addresses and job titles can change, so matching on those attributes could create duplicates or associate an update with the wrong person. I ran python -m source.hr.validate against the six-record feed and captured six valid records, zero errors and a PASS result.

I also tested missing and duplicate employee IDs. Each invalid feed produced five valid records, one error and a FAIL result, with error details written to validation_errors.jsonl.

Step 9 Review the first reconciliation plan

I ran python -m source.reconcile --dry-run before applying the initial load. Figure 4 shows six valid records, zero errors and six New classifications. The command reported NO WRITES PERFORMED.

```
(.venv) iga@iga-engine-01:~/iga-lab$ python -m source.reconcile --dry-run

HR SOURCE VALIDATION
--------------------
Source:        employees.csv
Valid records: 6
Errors:        0
Result:        PASS

RECONCILIATION PLAN
-------------------
[DRY-RUN] EMP001 Luke Skywalker — NEW
[DRY-RUN] EMP002 Leia Organa — NEW
[DRY-RUN] EMP003 Han Solo — NEW
[DRY-RUN] EMP004 Obi-Wan Kenobi — NEW
[DRY-RUN] EMP005 Anakin Skywalker — NEW
[DRY-RUN] EMP006 Ahsoka Tano — NEW

SUMMARY
-------
New:        6
Changed:    0
Unchanged:  0
Terminated: 0

DRY-RUN COMPLETE — NO WRITES PERFORMED
```

Figure 4. The initial dry run validates six synthetic identities and classifies all six as New before the apply step.

This checkpoint let me review the proposed identity changes before writing them. The no-write result is captured program output; independent before-and-after database snapshots were not captured.

Connector verification and outcome

Step 10 Apply and verify the initial records

I ran python -m source.reconcile --apply. After completion, I queried identities through the Python database connection and confirmed EMP001 through EMP006 were stored with active status. I also checked the successful reconciliation audit event and its matching correlation ID.

Step 11 Test repeat and changed input

I reran the comparison against an unchanged feed and received six Unchanged classifications. I then changed the synthetic EMP003 job title and ran another dry run. Figure 5 shows one Changed identity and five Unchanged identities.

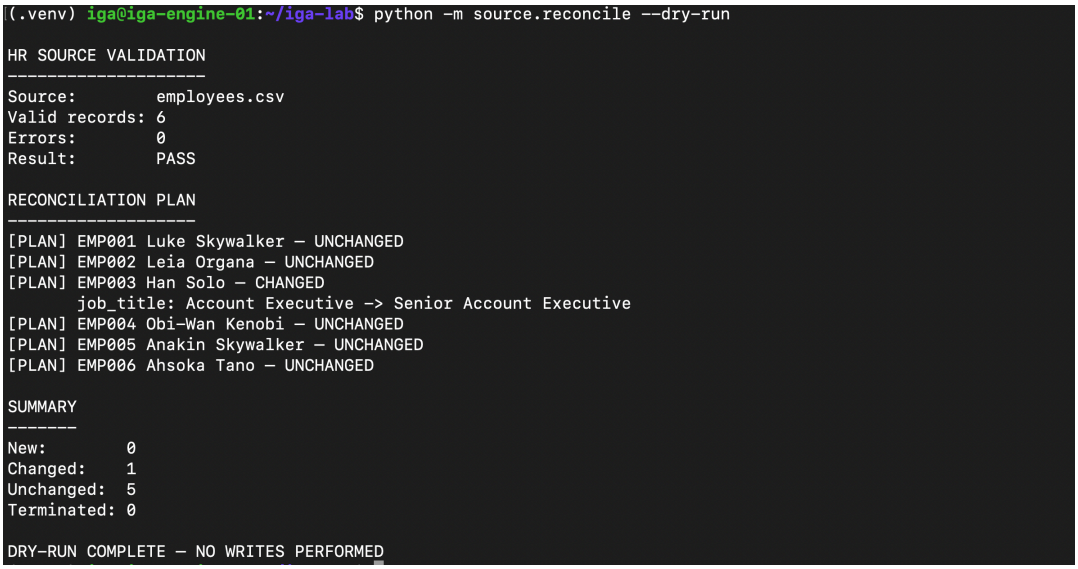


Figure 5. A job-title change is isolated to EMP003 while the other five identities remain Unchanged.

Step 12 Block an incomplete HR feed

I tested a five-record feed against the six-record minimum. The run was blocked: expected at least six, received five. This guard helps prevent a truncated HR export from being mistaken for mass departures. Earlier threshold wording still needs correction and recapture.

Result and remaining work

These checks established validated HR ingestion, saved identity records, change detection and traceable audit results. The job-title test covered source-data comparison. Downstream mover access changes and Okta connector setup belong to later workflow work.

Remaining checks include VM specifications, restore testing, effective database grants, audit-write protections, dependency locking and independent dry-run state comparisons.